Threaded BST Insertion

Thread Visualization, Boundary Insertions and Neighbor Verification

Data Structures Project Report • C++

1. Problem Statement

Suggested Modifications

- Display thread changes after every insertion.
- Insert values smaller than the minimum and larger than the maximum.
- Verify predecessor and successor results after insertion.

Objectives

- **Dynamic Thread Visualization:** Track and display precise thread pointer shifts, tracking assignments, and lthread/rthread flag states in real-time as nodes are appended.
- **Extremal Boundary Integration:** Validate structural stability and boundary preservation during insertions that extend past the active minimum and maximum key thresholds.
- **Automated Neighbor Verification:** Programmatically cross-verify node neighborhoods using a continuous execution checker to prove that every thread perfectly matches its strategic inorder predecessor or successor.
- **Eliminate Traversal Overheads:** Maintain structural stackless efficiency to run sequential O(n) printing routes using clean thread links without runtime stack layers.

2. Algorithm

1. Node Allocation: Allocate memory for node `tmp`. Assign `tmp->data = key`, `tmp->left = nullptr`, `tmp->right = nullptr`, and set both `lthread` and `rthread` flags to true.

2. Base Condition: If `root` equals `nullptr`, point `root` to `tmp`. Print root setup metrics, run `verifyThreads(tmp)`, and return `root`.

3. Traverse Matrix: Set crawler index `ptr = root` and companion pointer `parent = nullptr`.

4. Position Match Loop: Execute while `ptr != nullptr`:

- Set `parent = ptr`.
- If `key == ptr->data`, notify user of duplicate drop, clean up `tmp` memory, and return `root`.
- If `key < ptr->data`: If `ptr->lthread == false`, step left (`ptr = ptr->left`). Else, exit the loop.
- If `key > ptr->data`: If `ptr->rthread == false`, step right (`ptr = ptr->right`). Else, exit the loop.

5. Left Border Placement: If `key < parent->data`:

- Point `tmp->left` to `parent->left` (inherits predecessor context).
- Point `tmp->right` to `parent` (sets parent as its successor target).
- Toggle `parent->lthread = false` (parent left link converts into an explicit child node pointer).

- Assign `parent->left = tmp`.
- Print structural adjustments text block to screen.

6. Right Border Placement: If `key > parent->data`:

- Point `tmp->left` to `parent` (sets parent as its predecessor target).
- Point `tmp->right` to `parent->right` (inherits successor context).
- Toggle `parent->rthread = false` (parent right link converts into an explicit child node pointer).
- Assign `parent->right = tmp`.
- Print structural adjustments text block to screen.

7. Automated Validation Invocations:

- Invoke `verifyThreads(tmp, "Newly Inserted Node Validation")`.
- Invoke `verifyThreads(parent, "Parent Node Link Validation")`.

8. Exit Routine: Return the updated root memory address.

3. Flowchart

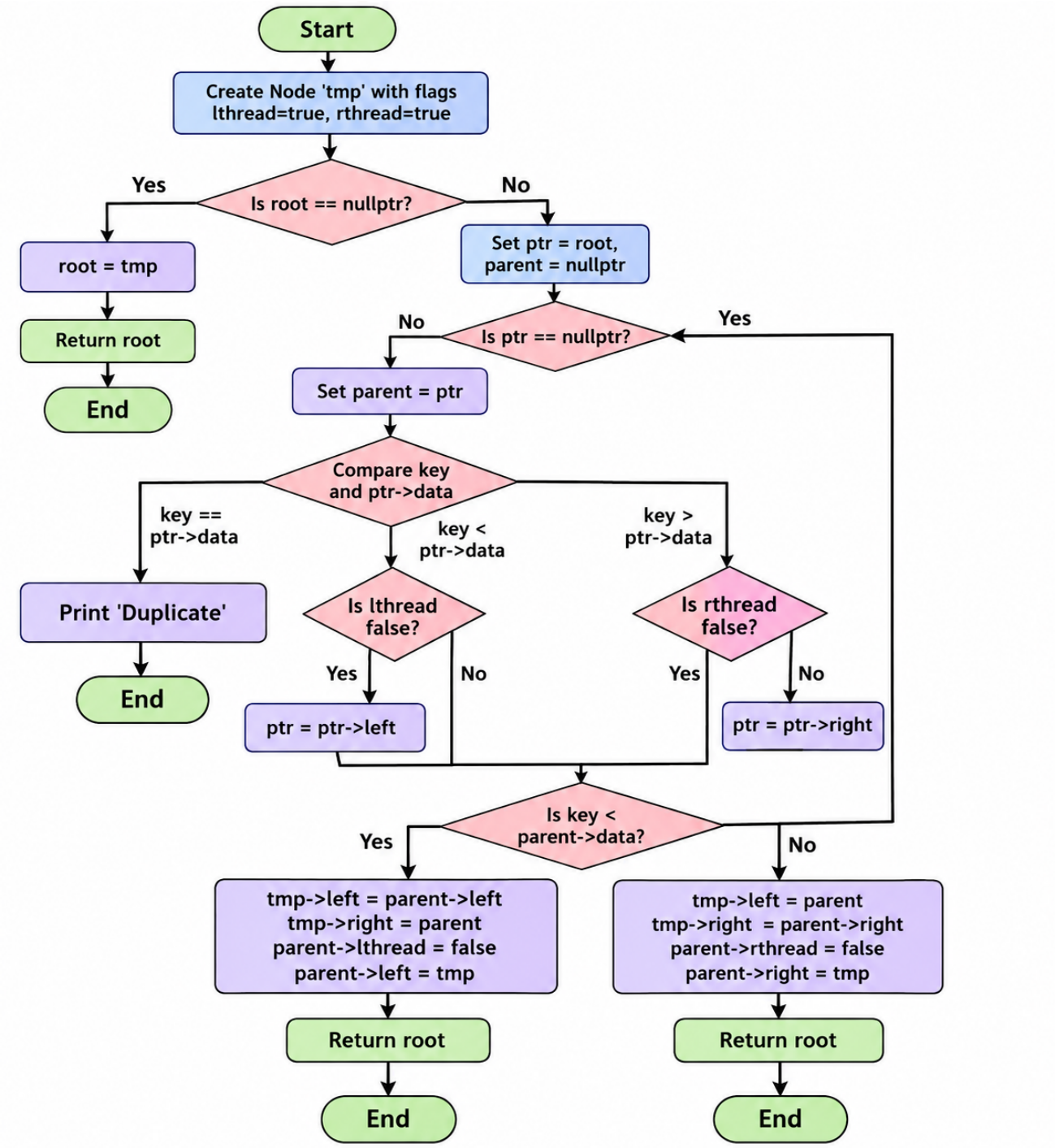


Figure 1. Flowchart of the insertion routine.

4. Code

```
#include <iostream>
#include <string>
using namespace std;
struct Node {
    int data;
    Node *left, *right;
    bool lthread;
    bool rthread;
};
string nodeVal(Node* n) {
    if (n == nullptr) return "NULL";
    return to_string(n->data);
}

Node* getMinNode(Node* root) {
    if (root == nullptr) return nullptr;
    Node* ptr = root;
    while (!ptr->lthread) {
        ptr = ptr->left;
    }
    return ptr;
}
Node* getMaxNode(Node* root) {
    if (root == nullptr) return nullptr;
    Node* ptr = root;
    while (!ptr->rthread) {
        ptr = ptr->right;
    }
    return ptr;
}
Node* inorderSuccessor(Node* ptr) {
    if (ptr == nullptr) return nullptr;
    if (ptr->rthread) {
        return ptr->right;
    }
    ptr = ptr->right;
    while (!ptr->lthread) {
        ptr = ptr->left;
    }
    return ptr;
}

Node* inorderPredecessor(Node* ptr) {
    if (ptr == nullptr) return nullptr;
    if (ptr->lthread) {
        return ptr->left;
    }
    ptr = ptr->left;
    while (!ptr->rthread) {
        ptr = ptr->right;
    }
    return ptr;
}

void verifyThreads(Node* target, string contextMessage) {
    if (target == nullptr) return;

    cout << "   [Verification - " << contextMessage << "]:\n";

    Node* pred = inorderPredecessor(target);
    Node* succ = inorderSuccessor(target);

    cout << "     * Node (" << target->data << ") Predecessor -> " << nodeVal(pred) << "\n";
    cout << "     * Node (" << target->data << ") Successor   -> " << nodeVal(succ) << "\n";

    if (target->lthread) {
        cout << "     * Verified: Left link is a thread pointing to Predecessor (" << nodeVal(target->left) << ")\n";
    }
    if (target->rthread) {
        cout << "     * Verified: Right link is a thread pointing to Successor (" << nodeVal(target->right) << ")\n";
    }
}
Node* insertAndTrack(Node* root, int key) {
    cout << "\n========================================\n";
    cout << "INSERTING KEY: " << key << "\n";
    cout << "========================================\n";

    Node* tmp = new Node();
    tmp->data = key;
    tmp->left = nullptr;
```

```
    tmp->right = nullptr;
    tmp->lthread = true;
    tmp->rthread = true;

    if (root == nullptr) {
        root = tmp;
        cout << "Tree was empty. Node " << key << " set as Root.\n";
        verifyThreads(tmp, "Root Creation");
        return root;
    }

    Node* ptr = root;
    Node* parent = nullptr;

    while (ptr != nullptr) {
        if (key == ptr->data) {
            cout << "Warning: Duplicate key " << key << " ignored.\n";
            delete tmp;
            return root;
        }

        parent = ptr;

        if (key < ptr->data) {
            if (!ptr->lthread) {
                ptr = ptr->left;
            } else {
                break;
            }
        } else {
            if (!ptr->rthread) {
                ptr = ptr->right;
            } else {
                break;
            }
        }
    }

    if (key < parent->data) {
        tmp->left = parent->left;
        tmp->right = parent;
        parent->lthread = false;
        parent->left = tmp;

        cout << "Attached " << key << " as LEFT child of " << parent->data << "\n";
        cout << "Thread Updates:\n";
        cout << " -> " << key << "->left  thread now points to Predecessor: " << nodeVal(tmp->left) << "\n";
        cout << " -> " << key << "->right thread now points to Successor:   " << nodeVal(tmp->right) << "\n";
        cout << " -> Parent " << parent->data << "->lthread set to FALSE (now structural link)\n";
    } else {
        tmp->left = parent;
        tmp->right = parent->right;
        parent->rthread = false;
        parent->right = tmp;

        cout << "Attached " << key << " as RIGHT child of " << parent->data << "\n";
        cout << "Thread Updates:\n";
        cout << " -> " << key << "->left  thread now points to Predecessor: " << nodeVal(tmp->left) << "\n";
        cout << " -> " << key << "->right thread now points to Successor:   " << nodeVal(tmp->right) << "\n";
        cout << " -> Parent " << parent->data << "->rthread set to FALSE (now structural link)\n";
    }

    verifyThreads(tmp, "Newly Inserted Node Validation");
    verifyThreads(parent, "Parent Node Link Validation");

    return root;
}
void printInorder(Node* root) {
    if (root == nullptr) {
        cout << "Tree is empty.\n";
        return;
    }
    Node* ptr = getMinNode(root);
    cout << "Current Inorder Sequence: ";
    while (ptr != nullptr) {
        cout << ptr->data << " ";
        ptr = inorderSuccessor(ptr);
    }
    cout << "\n";
}

int main() {
    Node* root = nullptr;
```

```
    root = insertAndTrack(root, 20);
    root = insertAndTrack(root, 10);
    root = insertAndTrack(root, 30);

    cout << "\n--- Base Tree Status ---\n";
    printInorder(root);

    root = insertAndTrack(root, 2);


    root = insertAndTrack(root, 45);

    cout << "\n--- Final Tree Status ---\n";
    printInorder(root);

    return 0;
}
```